

4교시: Inspect와 exec로 내부 확인

수업 목표

- docker inspect와 docker exec의 역할을 구분한다.
- port, image, restart policy 같은 metadata를 선별해서 확인한다.
- container 내부 filesystem/process를 직접 확인한다.

개념 설명

inspect는 Docker가 알고 있는 metadata를 보는 명령이다. port mapping, network, mount, image, env, restart policy처럼 container 외부 계약을 확인할 때 유용하다. exec는 이미 실행 중인 container 안에서 명령을 실행한다. filesystem, process, config file, network tool 존재 여부처럼 내부 상태를 볼 때 쓴다.

두 명령을 섞어 쓰면 안 된다. port mapping이 궁금하면 inspect, nginx가 실제로 어떤 파일을 serving 하는지 궁금하면 exec가 맞다.

단, inspect와 exec env는 secret을 보여줄 수 있다. --env-file .env로 넣은 값도 container metadata나 container 내부 환경에서 확인될 수 있다. 그래서 수업 기록에는 전체 출력 대신 필요한 key와 masking된 값만 남긴다.

실습 명령

```
docker inspect paperclip-day4-nginx --format 'Ports={{json .NetworkSettings.Ports}}'
docker inspect paperclip-day4-nginx --format 'Image={{.Config.Image}} Restart={{json .HostConfig.RestartPolicy}}'
docker exec paperclip-day4-nginx ls -l /usr/share/nginx/html
docker exec paperclip-day4-nginx sh -c 'ps | head'
```

Expected:

```
Ports={"80/tcp":[{"HostIp":"0.0.0.0","HostPort":"18084"}]}
Image=nginx:1.27-alpine
index.html
nginx
```

선택 기준

알고 싶은 것	먼저 쓸 명령
host port가 어디에 연결됐는가	<code>docker inspect ... NetworkSettings.Ports</code>
어떤 image로 뒀는가	<code>docker inspect ... Config.Image</code>
restart policy가 무엇인가	<code>docker inspect ... HostConfig.RestartPolicy</code>
container 안에 파일이 있는가	<code>docker exec ... ls</code>
process가 무엇인가	<code>docker exec ... ps</code>
env key가 적용됐는가	<code>docker inspect ... Config.Env</code> 또는 <code>docker exec ... env</code>

env 확인 시 masking

env 확인이 필요하면 key 이름과 적용 여부를 본다.

```
docker rm -f paperclip-day4-env-inspect || true
docker run -d --name paperclip-day4-env-inspect --env-file
week2/day4/labs/env-report/.env alpine:3.20 sleep 300
docker inspect paperclip-day4-env-inspect --format '{{range .Config.Env}}
{{println .}}{{end}}' | grep -E 'APP_ENV|FEATURE_FLAG|DB_PASSWORD'
docker exec paperclip-day4-env-inspect sh -c 'env | grep -E
"APP_ENV|FEATURE_FLAG|DB_PASSWORD" '
```

Expected:

```
APP_ENV=practice
FEATURE_FLAG=on
DB_PASSWORD=change-me-locally
```

제출물에는 다음처럼 남긴다.

```
APP_ENV=practice
FEATURE_FLAG=on
DB_PASSWORD=***masked***
```

해석: inspect와 exec env 모두 값 확인이 가능하다. 그래서 문제 해결에는 유용하지만, 제출물이나 질문 글에는 masking이 필요하다.

주의

`docker inspect` 전체 JSON을 README에 붙이면 읽기 어렵다. 문제와 관련된 field만 뽑아서 증거로 남긴다.

Kubernetes에서도 같은 기준이 이어진다. Pod의 `env`, `ConfigMap`, `Secret mount`를 확인할 수는 있지만, 민감한 값을 그대로 issue나 README에 붙이면 안 된다.

다음 연결

다음 교시는 resource 관찰과 restart policy를 다룬다.